OWASP Top 10 Vulnerabilities

Introduction

The **Open Web Application Security Project (OWASP)** is a nonprofit foundation that works to improve software security. One of its most recognized contributions is the **OWASP Top 10**, a regularly updated list highlighting the most critical security risks to web applications. The OWASP Top 10 serves as a standard awareness document for developers and organizations, helping them understand, prioritize, and mitigate prevalent security vulnerabilities. This assignment explores the latest OWASP Top 10 list (2021 version) and provides detailed explanations of five selected vulnerabilities.

The OWASP Top 10 (2021) List

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable and Outdated Components
7. Identification and Authentication Failures
8. Software and Data Integrity Failures
9. Security Logging and Monitoring Failures
10. Server-Side Request Forgery (SSRF)

Detailed Explanation of Five Vulnerabilities

1. Broken Access Control

Definition: Broken Access Control occurs when restrictions on what authenticated users are allowed to do are not properly enforced. This enables attackers to gain unauthorized access to sensitive data or functionality.

Examples:

- Accessing another user's account by modifying the user ID in the URL.
- Changing roles or permissions using client-side manipulation.
- Accessing admin functionality without appropriate permissions.

Impact: This is the most commonly found vulnerability, with severe consequences such as privilege escalation, data theft, and unauthorized modifications. Proper server-side access control checks are essential to mitigate this risk.

2. Cryptographic Failures

Definition: Previously known as "Sensitive Data Exposure," this category focuses on failures related to cryptography such as insecure algorithms, improper key management, and missing encryption.

Examples:

- Storing passwords using outdated hashing algorithms like MD5 or SHA-1.
- Transmitting sensitive data over unencrypted HTTP connections.
- Hardcoded or weak encryption keys.

Impact: Cryptographic failures can lead to the exposure of confidential data including passwords, credit card numbers, or personal identifiers. It is critical to use strong encryption standards, secure protocols, and best practices for storing and handling sensitive data.

3. Injection

Definition: Injection flaws occur when untrusted data is sent to an interpreter as part of a command or query, allowing attackers to execute unintended commands or access data without authorization.

Examples:

- SQL Injection: Malicious SQL statements that manipulate or access database content.
- Command Injection: Executing system-level commands via user input.

- Cross-Site Scripting (XSS): Injecting malicious scripts into webpages.

Impact: Injection vulnerabilities can result in data leaks, authentication bypass, data corruption, or even complete system compromise. Preventing injection requires proper input validation, parameterized queries, and safe API usage.

4. Insecure Design

Definition: Insecure Design refers to flaws in the system's architecture and design that create inherent vulnerabilities. It highlights the importance of proactive security measures during the planning and design phases of software development.

Examples:

- Absence of threat modeling and secure design practices.
- Features implemented without considering abuse cases.
- Inadequate segregation of roles or access pathways.

Impact: Even if implementation is done correctly, insecure design may leave critical gaps in system security. It requires embedding security principles early in the Software Development Life Cycle (SDLC).

5. Security Misconfiguration

Definition: This vulnerability occurs due to improper implementation or oversight in configuring the application environment, servers, databases, or frameworks.

Examples:

- Leaving default credentials active.
- Exposing development/debugging information in production.
- Running applications with unnecessary features enabled.

Impact: Misconfigurations can expose systems to attacks, increase the attack surface, and often allow attackers to access sensitive data or functionality. Regular audits and hardened configurations help reduce this risk.